HumanEvalPack — 30 Tasks

Python & C++ · Organizado por dificuldade

Dataset: bigcode/humanevalpack

FÁCEIS

Python/23 — STRLEN

Docstring:

```
Return length of given string
>>> strlen('')
0
>>> strlen('abc')
3
```

Python:

```
    return len(string)
```

C++:

```
#include<stdio.h>
#include<math.h>
#include<string>
using namespace std;
#include<algorithm>
#include<stdlib.h>
int strlen(string str){
    return str.length();
}
```

Python/54 — SAME_CHARS

Docstring:

```
Check if two words have the same characters.
>>> same_chars('eabcdzzzz', 'dddzzzzzzzddeddabc')
True
>>> same_chars('abcd', 'dddddddabc')
True
>>> same_chars('dddddddabc', 'abcd')
True
>>> same_chars('eabcd', 'dddddddabc')
False
>>> same_chars('abcd', 'dddddddabce')
```

```
False
>>> same_chars('eabcdzzzz', 'dddzzzzzzzddddabc')
False
```

Python:

```
    return set(s0) == set(s1)
```

C++:

```
#include<stdio.h>
#include<math.h>
#include<string>
#include<algorithm>
using namespace std;
#include<stdlib.h>
bool same_chars(string s0,string s1){
    for (int i=0;i<s0.length();i++)
    if (find(s1.begin(),s1.end(),s0[i])==s1.end())
        return false;
    for (int i=0;i<s1.length();i++)
    if (find(s0.begin(),s0.end(),s1[i])==s0.end())
        return false;
    return true;
}
```

Python/35 — MAX_ELEMENT

Docstring:

```
Return maximum element in the list.
>>> max_element([1, 2, 3])
3
>>> max_element([5, 3, -5, 2, -3, 3, 9, 0, 123, 1, -10])
123
```

Python:

```
    m = l[0]
    for e in l:
        if e > m:
            m = e
    return m
```

C++:

```
#include<stdio.h>
#include<math.h>
#include<vector>
#include<algorithm>
using namespace std;
#include<stdlib.h>
float max_element(vector<float> l){
```

```
  float max=-10000;
  for (int i=0;i<l.size();i++)
  if (max<l[i]) max=l[i];
  return max;

}
```

Python/27 — FLIP_CASE

Docstring:

```
For a given string, flip lowercase characters to uppercase and uppercase to
lowercase.
>>> flip_case('Hello')
'hELLO'
```

Python:

```
    return string.swapcase()
```

C++:

```
#include<stdio.h>
#include<math.h>
#include<string>
using namespace std;
#include<algorithm>
#include<stdlib.h>
string flip_case(string str){
    string out="";
    for (int i=0;i<str.length();i++)
    {
        char w=str[i];
        if (w>=97 and w<=122) {w-=32;}
        else
            if (w>=65 and w<=90){ w+=32;}
        out=out+w;
    }
    return out;
}
```

Python/52 — BELOW_THRESHOLD

Docstring:

```
Return True if all numbers in the list l are below threshold t.
>>> below_threshold([1, 2, 4, 10], 100)
True
>>> below_threshold([1, 20, 4, 10], 5)
False
```

Python:

```
    for e in l:
        if e >= t:
            return False
    return True
```

C++:

```
#include<stdio.h>
#include<math.h>
#include<vector>
using namespace std;
#include<algorithm>
#include<stdlib.h>
bool below_threshold(vector<int>l, int t){
    for (int i=0;i<l.size();i++)
        if (l[i]>=t) return false;
    return true;
}
```

Python/3 — BELOW_ZERO

Docstring:

```
You're given a list of deposit and withdrawal operations on a bank account
that starts with
zero balance. Your task is to detect if at any point the balance of account
fallls below zero, and
at that point function should return True. Otherwise it should return False.
>>> below_zero([1, 2, 3])
False
>>> below_zero([1, 2, -4, 5])
True
```

Python:

```
    balance = 0

    for op in operations:
        balance += op
        if balance < 0:
            return True

    return False
```

C++:

```
#include<stdio.h>
#include<vector>
using namespace std;
#include<algorithm>
#include<math.h>
#include<stdlib.h>
bool below_zero(vector<int> operations){
```

```
    int num=0;
    for (int i=0;i<operations.size();i++)
    {
        num+=operations[i];
        if (num<0) return true;
    }
    return false;
}
```

Python/6 — PARSE_NESTED_PARENS

Docstring:

```
Input to this function is a string represented multiple groups for nested
parentheses separated by spaces.
For each of the group, output the deepest level of nesting of parentheses.
E.g. (()()) has maximum two levels of nesting while ((())) has three.
>>> parse_nested_parens('(()()) ((())) () ((())()())')
[2, 3, 1, 3]
```

Python:

```
    def parse_paren_group(s):
        depth = 0
        max_depth = 0
        for c in s:
            if c == '(':
                depth += 1
                max_depth = max(depth, max_depth)
            else:
                depth -= 1

        return max_depth

    return [parse_paren_group(x) for x in paren_string.split(' ') if x]
```

C++:

```
#include<stdio.h>
#include<vector>
#include<string>
using namespace std;
#include<algorithm>
#include<math.h>
#include<stdlib.h>
vector<int> parse_nested_parens(string paren_string){
    vector<int> all_levels;
    string current_paren;
    int level=0,max_level=0;
    char chr;
    int i;
    for (i=0;i<paren_string.length();i++)
    {
```

```
        chr=paren_string[i];
        if (chr=='(')
        {
        level+=1;
        if (level>max_level) max_level=level;
        current_paren+=chr;
        }
        if (chr==')')
        {
            level-=1;
            current_paren+=chr;
            if (level==0){
                all_levels.push_back(max_level);
                current_paren="";
                max_level=0;
            }
        }
    }
    return all_levels;
}
```

Python/14 — ALL_PREFIXES

Docstring:

```
Return list of all prefixes from shortest to longest of the input string
>>> all_prefixes('abc')
['a', 'ab', 'abc']
```

Python:

```
    result = []

    for i in range(len(string)):
        result.append(string[:i+1])
    return result
```

C++:

```
#include<stdio.h>
#include<vector>
#include<string>
using namespace std;
#include<algorithm>
#include<math.h>
#include<stdlib.h>
vector<string> all_prefixes(string str){
    vector<string> out;
    string current="";
    for (int i=0;i<str.length();i++)
    {
        current=current+str[i];
        out.push_back(current);
```

```
    }
    return out;
}
```

Python/30 — GET_POSITIVE

Docstring:

```
Return only positive numbers in the list.
>>> get_positive([-1, 2, -4, 5, 6])
[2, 5, 6]
>>> get_positive([5, 3, -5, 2, -3, 3, 9, 0, 123, 1, -10])
[5, 3, 2, 3, 9, 123, 1]
```

Python:

```
    return [e for e in l if e > 0]
```

C++:

```
#include<stdio.h>
#include<math.h>
#include<vector>
using namespace std;
#include<algorithm>
#include<stdlib.h>
vector<float> get_positive(vector<float> l){
    vector<float> out={};
    for (int i=0;i<l.size();i++)
        if (l[i]>0) out.push_back(l[i]);
    return out;
}
```

Python/26 — REMOVE_DUPLICATES

Docstring:

```
From a list of integers, remove all elements that occur more than once.
Keep order of elements left the same as in the input.
>>> remove_duplicates([1, 2, 3, 2, 4])
[1, 3, 4]
```

Python:

```
    import collections
    c = collections.Counter(numbers)
    return [n for n in numbers if c[n] <= 1]
```

C++:

```
#include<stdio.h>
```

```
#include<math.h>
#include<vector>
#include<algorithm>
using namespace std;
#include<stdlib.h>
vector<int> remove_duplicates(vector<int> numbers){
    vector<int> out={};
    vector<int> has1={};
    vector<int> has2={};
    for (int i=0;i<numbers.size();i++)
    {
        if (find(has2.begin(),has2.end(),numbers[i])!=has2.end()) continue;
        if (find(has1.begin(),has1.end(),numbers[i])!=has1.end())
        {

            has2.push_back(numbers[i]);
        }
        else has1.push_back(numbers[i]);
    }
    for (int i=0;i<numbers.size();i++)
    if (find(has2.begin(),has2.end(),numbers[i])==has2.end())
        out.push_back(numbers[i]);
    return out;


}
```

MÉDIOS

Python/13 — GREATEST_COMMON_DIVISOR

Docstring:

```
Return a greatest common divisor of two integers a and b
>>> greatest_common_divisor(3, 5)
1
>>> greatest_common_divisor(25, 15)
5
```

Python:

```
    while b:
        a, b = b, a % b
    return a
```

C++:

```
#include<stdio.h>
using namespace std;
#include<algorithm>
#include<math.h>
#include<stdlib.h>
int greatest_common_divisor(int a, int b){
    int out,m;
    while (true){
        if (a<b)
        {
            m=a;a=b;b=m;
        }
        a=a%b;
        if (a==0) return b;
    }
}
```

Python/25 — FACTORIZE

Docstring:

```
Return list of prime factors of given integer in the order from smallest to
largest.
Each of the factors should be listed number of times corresponding to how
many times it appeares in factorization.
Input number should be equal to the product of all factors
>>> factorize(8)
[2, 2, 2]
>>> factorize(25)
[5, 5]
>>> factorize(70)
```

```
[2, 5, 7]
```

Python:

```
    import math
    fact = []
    i = 2
    while i <= int(math.sqrt(n) + 1):
        if n % i == 0:
            fact.append(i)
            n //= i
        else:
            i += 1

    if n > 1:
        fact.append(n)
    return fact
```

C++:

```
#include<stdio.h>
#include<math.h>
#include<vector>
using namespace std;
#include<algorithm>
#include<stdlib.h>
vector<int> factorize(int n){
    vector<int> out={};
    for (int i=2;i*i<=n;i++)
        if (n%i==0)
        {
            n=n/i;
            out.push_back(i);
            i-=1;
        }
    out.push_back(n);
    return out;
}
```

Python/36 — FIZZ_BUZZ

Docstring:

```
Return the number of times the digit 7 appears in integers less than n which
are divisible by 11 or 13.
>>> fizz_buzz(50)
0
>>> fizz_buzz(78)
2
>>> fizz_buzz(79)
3
```

Python:

```
    ns = []
    for i in range(n):
        if i % 11 == 0 or i % 13 == 0:
            ns.append(i)
    s = ''.join(list(map(str, ns)))
    ans = 0
    for c in s:
        ans += (c == '7')
    return ans
```

C++:

```
#include<stdio.h>
#include<math.h>
using namespace std;
#include<algorithm>
#include<stdlib.h>
int fizz_buzz(int n){
    int count=0;
    for (int i=0;i<n;i++)
    if (i%11==0 or i%13==0)
    {
        int q=i;
        while (q>0)
        {
            if (q%10==7) count+=1;
            q=q/10;
        }
    }
    return count;
}
```

Python/72 — WILL_IT_FLY

Docstring:

```
Write a function that returns True if the object q will fly, and False
otherwise.
The object q will fly if it's balanced (it is a palindromic list) and the sum
of its elements is less than or equal the maximum possible weight w.
Example:
will_it_fly([1, 2], 5) ➞ False
# 1+2 is less than the maximum possible weight, but it's unbalanced.
will_it_fly([3, 2, 3], 1) ➞ False
# it's balanced, but 3+2+3 is more than the maximum possible weight.
will_it_fly([3, 2, 3], 9) ➞ True
# 3+2+3 is less than the maximum possible weight, and it's balanced.
will_it_fly([3], 5) ➞ True
# 3 is less than the maximum possible weight, and it's balanced.
```

Python:

```
    if sum(q) > w:
        return False
```

```
    i, j = 0, len(q)-1
    while i<j:
        if q[i] != q[j]:
            return False
        i+=1
        j-=1
    return True
```

C++:

```
#include<stdio.h>
#include<math.h>
#include<vector>
using namespace std;
#include<algorithm>
#include<stdlib.h>
bool will_it_fly(vector<int> q,int w){
    int sum=0;
    for (int i=0;i<q.size();i++)
    {
        if (q[i]!=q[q.size()-1-i]) return false;
        sum+=q[i];
    }
    if (sum>w) return false;
    return true;
}
```

Python/38 — DECODE_CYCLIC

Docstring:

```
takes as input string encoded with encode_cyclic function. Returns decoded
string.
```

Python:

```
    return encode_cyclic(encode_cyclic(s))
```

C++:

```
#include<stdio.h>
#include<math.h>
#include<string>
using namespace std;
#include<algorithm>
#include<stdlib.h>
string encode_cyclic(string s){
    int l=s.length();
    int num=(l+2)/3;
    string x,output;
    int i;
    for (i=0;i*3<l;i++)
```

```
    {
        x=s.substr(i*3,3);
        if (x.length()==3) x=x.substr(1)+x[0];
        output=output+x;
    }
    return output;
}


string decode_cyclic(string s){
   int l=s.length();
    int num=(l+2)/3;
    string x,output;
    int i;
    for (i=0;i*3<l;i++)
    {
    int l=s.length();
    int num=(l+2)/3;
    string x,output;
    int i;
    for (i=0;i*3<l;i++)
    {
        x=s.substr(i*3,3);
        if (x.length()==3) x=x[2]+x.substr(0,2);
        output=output+x;
    }
    return output;


}
```

Python/10 — MAKE_PALINDROME

Docstring:

```
Find the shortest palindrome that begins with a supplied string.
Algorithm idea is simple:
- Find the longest postfix of supplied string that is a palindrome.
- Append to the end of the string reverse of a string prefix that comes
before the palindromic suffix.
>>> make_palindrome('')
''
>>> make_palindrome('cat')
'catac'
>>> make_palindrome('cata')
'catac'
```

Python:

```
    if not string:
        return ''

    beginning_of_suffix = 0
```

```
    while not is_palindrome(string[beginning_of_suffix:]):
        beginning_of_suffix += 1

    return string + string[:beginning_of_suffix][::-1]
```

C++:

```
#include<stdio.h>
#include<string>
using namespace std;
#include<algorithm>
#include<math.h>
#include<stdlib.h>
bool is_palindrome(string str){
    string s(str.rbegin(),str.rend());
    return s==str;
}
string make_palindrome(string str){
   int i;
   for (i=0;i<str.length();i++)
   {
        string rstr=str.substr(i);
        if (is_palindrome(rstr))
        {
            string nstr;
            nstr=str.substr(0,i);
            string n2str(nstr.rbegin(),nstr.rend());
            return str+n2str;
        }
   }
   string n2str(str.rbegin(),str.rend());
   return str+n2str;
}
```

Python/40 — TRIPLES_SUM_TO_ZERO

Docstring:

```
triples_sum_to_zero takes a list of integers as an input.
it returns True if there are three distinct elements in the list that
sum to zero, and False otherwise.
>>> triples_sum_to_zero([1, 3, 5, 0])
False
>>> triples_sum_to_zero([1, 3, -2, 1])
True
>>> triples_sum_to_zero([1, 2, 3, 7])
False
>>> triples_sum_to_zero([2, 4, -5, 3, 9, 7])
True
>>> triples_sum_to_zero([1])
False
```

Python:

```
    for i in range(len(l)):
        for j in range(i + 1, len(l)):
            for k in range(j + 1, len(l)):
                if l[i] + l[j] + l[k] == 0:
                    return True
    return False
```

C++:

```
#include<stdio.h>
#include<math.h>
#include<vector>
#include<algorithm>
using namespace std;
#include<stdlib.h>
bool triples_sum_to_zero(vector<int> l){
    for (int i=0;i<l.size();i++)
    for (int j=i+1;j<l.size();j++)
    for (int k=j+1;k<l.size();k++)
        if (l[i]+l[j]+l[k]==0) return true;
    return false;
}
```

Python/60 — SUM_TO_N

Docstring:

```
sum_to_n is a function that sums numbers from 1 to n.
>>> sum_to_n(30)
465
>>> sum_to_n(100)
5050
>>> sum_to_n(5)
15
>>> sum_to_n(10)
55
>>> sum_to_n(1)
1
```

Python:

```
    return sum(range(n + 1))
```

C++:

```
#include<stdio.h>
#include<math.h>
using namespace std;
#include<algorithm>
#include<stdlib.h>
int sum_to_n(int n){
    return n*(n+1)/2;
```

```
}
```

Python/62 — DERIVATIVE

Docstring:

```
xs represent coefficients of a polynomial.
xs[0] + xs[1] * x + xs[2] * x^2 + ....
Return derivative of this polynomial in the same form.
>>> derivative([3, 1, 2, 4, 5])
[1, 4, 12, 20]
>>> derivative([1, 2, 3])
[2, 6]
```

Python:

```
    return [(i * x) for i, x in enumerate(xs)][1:]
```

C++:

```
#include<stdio.h>
#include<math.h>
#include<vector>
using namespace std;
#include<algorithm>
#include<stdlib.h>
vector<float> derivative(vector<float> xs){
    vector<float> out={};
    for (int i=1;i<xs.size();i++)
        out.push_back(i*xs[i]);
    return out;
}
```

Python/74 — TOTAL_MATCH

Docstring:

```
Write a function that accepts two lists of strings and returns the list that
has
total number of chars in the all strings of the list less than the other
list.
if the two lists have the same number of chars, return the first list.
Examples
total_match([], []) ➞ []
total_match(['hi', 'admin'], ['hI', 'Hi']) ➞ ['hI', 'Hi']
total_match(['hi', 'admin'], ['hi', 'hi', 'admin', 'project']) ➞ ['hi',
'admin']
total_match(['hi', 'admin'], ['hI', 'hi', 'hi']) ➞ ['hI', 'hi', 'hi']
total_match(['4'], ['1', '2', '3', '4', '5']) ➞ ['4']
```

Python:

```
    l1 = 0
    for st in lst1:
        l1 += len(st)

    l2 = 0
    for st in lst2:
        l2 += len(st)

    if l1 <= l2:
        return lst1
    else:
        return lst2
```

C++:

```
#include<stdio.h>
#include<math.h>
#include<vector>
#include<string>
#include<algorithm>
using namespace std;
#include<stdlib.h>
vector<string> total_match(vector<string> lst1,vector<string> lst2){
    int num1,num2,i;
    num1=0;num2=0;
    for (i=0;i<lst1.size();i++)
        num1+=lst1[i].length();
    for (i=0;i<lst2.size();i++)
        num2+=lst2[i].length();
    if (num1>num2) return lst2;
    return lst1;
}
```

DIFÍCEIS

Python/119 — MATCH_PARENS

Docstring:

```
You are given a list of two strings, both strings consist of open
parentheses '(' or close parentheses ')' only.
Your job is to check if it is possible to concatenate the two strings in
some order, that the resulting string will be good.
A string S is considered to be good if and only if all parentheses in S
are balanced. For example: the string '(())()' is good, while the string
'())' is not.
Return 'Yes' if there's a way to make a good string, and return 'No'
otherwise.
Examples:
match_parens(['()(', ')']) == 'Yes'
match_parens([')', ')']) == 'No'
```

Python:

```
    def check(s):
        val = 0
        for i in s:
            if i == '(':
                val = val + 1
            else:
                val = val - 1
            if val < 0:
                return False
        return True if val == 0 else False

    S1 = lst[0] + lst[1]
    S2 = lst[1] + lst[0]
    return 'Yes' if check(S1) or check(S2) else 'No'
```

C++:

```
#include<stdio.h>
#include<math.h>
#include<vector>
#include<string>
using namespace std;
#include<algorithm>
#include<stdlib.h>
string match_parens(vector<string> lst){
    string l1=lst[0]+lst[1];
    int i,count=0;
    bool can=true;
    for (i=0;i<l1.length();i++)
        {
            if (l1[i]=='(') count+=1;
            if (l1[i]==')') count-=1;
```

```
            if (count<0) can=false;
        }
    if (count!=0) return "No";
    if (can==true) return "Yes";
    l1=lst[1]+lst[0];
    can=true;
    for (i=0;i<l1.length();i++)
        {
            if (l1[i]=='(') count+=1;
            if (l1[i]==')') count-=1;
            if (count<0) can=false;
        }
    if (can==true) return "Yes";
    return "No";
}
```

Python/120 — MAXIMUM

Docstring:

```
Given an array arr of integers and a positive integer k, return a sorted list
of length k with the maximum k numbers in arr.
Example 1:
Input: arr = [-3, -4, 5], k = 3
Output: [-4, -3, 5]
Example 2:
Input: arr = [4, -4, 4], k = 2
Output: [4, 4]
Example 3:
Input: arr = [-3, 2, 1, 2, -1, -2, 1], k = 1
Output: [2]
Note:
1. The length of the array will be in the range of [1, 1000].
2. The elements in the array will be in the range of [-1000, 1000].
3. 0 <= k <= len(arr)
```

Python:

```
    if k == 0:
        return []
    arr.sort()
    ans = arr[-k:]
    return ans
```

C++:

```
#include<stdio.h>
#include<math.h>
#include<vector>
#include<algorithm>
using namespace std;
#include<stdlib.h>
vector<int> maximum(vector<int> arr,int k){
```

```
    sort(arr.begin(),arr.end());
    vector<int> out(arr.end()-k,arr.end());
    return out;
}
```

Python/140 — FIX_SPACES

Docstring:

```
Given a string text, replace all spaces in it with underscores,
and if a string has more than 2 consecutive spaces,
then replace all consecutive spaces with -
fix_spaces("Example") == "Example"
fix_spaces("Example 1") == "Example_1"
fix_spaces(" Example 2") == "_Example_2"
fix_spaces(" Example   3") == "_Example-3"
```

Python:

```
    new_text = ""
    i = 0
    start, end = 0, 0
    while i < len(text):
        if text[i] == " ":
            end += 1
        else:
            if end - start > 2:
                new_text += "-"+text[i]
            elif end - start > 0:
                new_text += "_"*(end - start)+text[i]
            else:
                new_text += text[i]
            start, end = i+1, i+1
        i+=1
    if end - start > 2:
        new_text += "-"
    elif end - start > 0:
        new_text += "_"
    return new_text
```

C++:

```
#include<stdio.h>
#include<string>
using namespace std;
#include<algorithm>
#include<math.h>
#include<stdlib.h>
string fix_spaces(string text){
    string out="";
    int spacelen=0;
    for (int i=0;i<text.length();i++)
    if (text[i]==' ') spacelen+=1;
```

```
    else
    {
        if (spacelen==1) out=out+'_';
        if (spacelen==2) out=out+"__";
        if (spacelen>2) out=out+'-';
        spacelen=0;
        out=out+text[i];
    }
    if (spacelen==1) out=out+'_';
    if (spacelen==2) out=out+"__";
    if (spacelen>2) out=out+'-';
    return out;
}
```

Python/145 — ORDER_BY_POINTS

Docstring:

```
Write a function which sorts the given list of integers
in ascending order according to the sum of their digits.
Note: if there are several items with similar sum of their digits,
order them based on their index in original list.
For example:
>>> order_by_points([1, 11, -1, -11, -12]) == [-1, -11, 1, -12, 11]
>>> order_by_points([]) == []
```

Python:

```
    def digits_sum(n):
        neg = 1
        if n < 0: n, neg = -1 * n, -1
        n = [int(i) for i in str(n)]
        n[0] = n[0] * neg
        return sum(n)
    return sorted(nums, key=digits_sum)
```

C++:

```
#include<stdio.h>
#include<math.h>
#include<vector>
#include<string>
#include<algorithm>
using namespace std;
#include<stdlib.h>
vector<int> order_by_points(vector<int> nums){
    vector<int> sumdigit={};
    for (int i=0;i<nums.size();i++)
    {
        string w=to_string(abs(nums[i]));
        int sum=0;
        for (int j=1;j<w.length();j++)
            sum+=w[j]-48;
```

```
        if (nums[i]>0) sum+=w[0]-48;
        else sum-=w[0]-48;
        sumdigit.push_back(sum);
    }
    int m;
    for (int i=0;i<nums.size();i++)
    for (int j=1;j<nums.size();j++)
    if (sumdigit[j-1]>sumdigit[j])
    {
        m=sumdigit[j];sumdigit[j]=sumdigit[j-1];sumdigit[j-1]=m;
        m=nums[j];nums[j]=nums[j-1];nums[j-1]=m;
    }

    return nums;
}
```

Python/159 — EAT

Docstring:

```
You're a hungry rabbit, and you already have eaten a certain number of
carrots,
but now you need to eat more carrots to complete the day's meals.
you should return an array of [ total number of eaten carrots after your
meals,
the number of carrots left after your meals ]
if there are not enough remaining carrots, you will eat all remaining
carrots, but will still be hungry.
Example:
* eat(5, 6, 10) -> [11, 4]
* eat(4, 8, 9) -> [12, 1]
* eat(1, 10, 10) -> [11, 0]
* eat(2, 11, 5) -> [7, 0]
Variables:
@number : integer
the number of carrots that you have eaten.
@need : integer
the number of carrots that you need to eat.
@remaining : integer
the number of remaining carrots thet exist in stock
Constrain:
* 0 <= number <= 1000
* 0 <= need <= 1000
* 0 <= remaining <= 1000
Have fun :)
```

Python:

```
    if(need <= remaining):
        return [ number + need , remaining-need ]
    else:
        return [ number + remaining , 0]
```

C++:

```
#include<stdio.h>
#include<vector>
using namespace std;
#include<algorithm>
#include<math.h>
#include<stdlib.h>
vector<int> eat(int number,int need,int remaining){
    if (need>remaining) return {number+remaining, 0};
    return {number+need,remaining-need};
}
```

Python/84 — SOLVE

Docstring:

```
Given a positive integer N, return the total sum of its digits in binary.
Example
For N = 1000, the sum of digits will be 1 the output should be "1".
For N = 150, the sum of digits will be 6 the output should be "110".
For N = 147, the sum of digits will be 12 the output should be "1100".
Variables:
@N integer
Constraints: 0 ≤ N ≤ 10000.
Output:
a string of binary number
```

Python:

```
    return bin(sum(int(i) for i in str(N)))[2:]
```

C++:

```
#include<stdio.h>
#include<math.h>
#include<string>
using namespace std;
#include<algorithm>
#include<stdlib.h>
string solve(int N){
    string str,bi="";
    str=to_string(N);
    int i,sum=0;
    for (int i=0;i<str.length();i++)
        sum+=str[i]-48;
    while (sum>0)
    {
        bi=to_string(sum%2)+bi;
        sum=sum/2;
    }
    return bi;
}
```

Python/105 — BY_LENGTH

Docstring:

```
Given an array of integers, sort the integers that are between 1 and 9
inclusive,
reverse the resulting array, and then replace each digit by its corresponding
name from
"One", "Two", "Three", "Four", "Five", "Six", "Seven", "Eight", "Nine".
For example:
arr = [2, 1, 1, 4, 5, 8, 2, 3]
-> sort arr -> [1, 1, 2, 2, 3, 4, 5, 8]
-> reverse arr -> [8, 5, 4, 3, 2, 2, 1, 1]
return ["Eight", "Five", "Four", "Three", "Two", "Two", "One", "One"]
If the array is empty, return an empty array:
arr = []
return []
If the array has any strange number ignore it:
arr = [1, -1 , 55]
-> sort arr -> [-1, 1, 55]
-> reverse arr -> [55, 1, -1]
return = ['One']
```

Python:

```python
    dic = {
        1: "One",
        2: "Two",
        3: "Three",
        4: "Four",
        5: "Five",
        6: "Six",
        7: "Seven",
        8: "Eight",
        9: "Nine",
    }
    sorted_arr = sorted(arr, reverse=True)
    new_arr = []
    for var in sorted_arr:
        try:
            new_arr.append(dic[var])
        except:
            pass
    return new_arr
```

C++:

```cpp
#include<stdio.h>
#include<math.h>
#include<vector>
#include<string>
#include<map>
#include<algorithm>
```

```
using namespace std;
#include<stdlib.h>
vector<string> by_length(vector<int> arr){
    map<int,string>
numto={{0,"Zero"},{1,"One"},{2,"Two"},{3,"Three"},{4,"Four"},{5,"Five"},{6,"Six"},{7,"
Seven"},{8,"Eight"},{9,"Nine"}};
    sort(arr.begin(),arr.end());
    vector<string> out={};
    for (int i=arr.size()-1;i>=0;i-=1)
        if (arr[i]>=1 and arr[i]<=9)
        out.push_back(numto[arr[i]]);
    return out;
}
```

Python/126 — IS_SORTED

Docstring:

```
Given a list of numbers, return whether or not they are sorted
in ascending order. If list has more than 1 duplicate of the same
number, return False. Assume no negative numbers and only integers.
Examples
is_sorted([5]) ➞ True
is_sorted([1, 2, 3, 4, 5]) ➞ True
is_sorted([1, 3, 2, 4, 5]) ➞ False
is_sorted([1, 2, 3, 4, 5, 6]) ➞ True
is_sorted([1, 2, 3, 4, 5, 6, 7]) ➞ True
is_sorted([1, 3, 2, 4, 5, 6, 7]) ➞ False
is_sorted([1, 2, 2, 3, 3, 4]) ➞ True
is_sorted([1, 2, 2, 2, 3, 4]) ➞ False
```

Python:

```
    count_digit = dict([(i, 0) for i in lst])
    for i in lst:
        count_digit[i]+=1
    if any(count_digit[i] > 2 for i in lst):
        return False
    if all(lst[i-1] <= lst[i] for i in range(1, len(lst))):
        return True
    else:
        return False
```

C++:

```
#include<stdio.h>
#include<math.h>
#include<vector>
#include<algorithm>
using namespace std;
#include<stdlib.h>
bool is_sorted(vector<int> lst){
    for (int i=1;i<lst.size();i++)
    {
```

```
        if (lst[i]<lst[i-1]) return false;
        if (i>=2 and lst[i]==lst[i-1] and lst[i]==lst[i-2]) return false;
    }
    return true;
}
```

Python/130 — TRI

Docstring:

```
Everyone knows Fibonacci sequence, it was studied deeply by mathematicians in
the last couple centuries. However, what people don't know is Tribonacci
sequence.
Tribonacci sequence is defined by the recurrence:
tri(1) = 3
tri(n) = 1 + n / 2, if n is even.
tri(n) =  tri(n - 1) + tri(n - 2) + tri(n + 1), if n is odd.
For example:
tri(2) = 1 + (2 / 2) = 2
tri(4) = 3
tri(3) = tri(2) + tri(1) + tri(4)
= 2 + 3 + 3 = 8
You are given a non-negative integer number n, you have to a return a list of
the
first n + 1 numbers of the Tribonacci sequence.
Examples:
tri(3) = [1, 3, 2, 8]
```

Python:

```
    if n == 0:
        return [1]
    my_tri = [1, 3]
    for i in range(2, n + 1):
        if i % 2 == 0:
            my_tri.append(i / 2 + 1)
        else:
            my_tri.append(my_tri[i - 1] + my_tri[i - 2] + (i + 3) / 2)
    return my_tri
```

C++:

```
#include<stdio.h>
#include<math.h>
#include<vector>
#include<algorithm>
using namespace std;
#include<stdlib.h>
vector<int> tri(int n){
    vector<int> out={1,3};
    if (n==0) return {1};
    for (int i=2;i<=n;i++)
    {
        if (i%2==0) out.push_back(1+i/2);
```

```
        else out.push_back(out[i-1]+out[i-2]+1+(i+1)/2);
    }
    return out;
}
```

Python/153 — STRONGEST_EXTENSION

Docstring:

```
You will be given the name of a class (a string) and a list of extensions.
The extensions are to be used to load additional classes to the class. The
strength of the extension is as follows: Let CAP be the number of the
uppercase
letters in the extension's name, and let SM be the number of lowercase
letters
in the extension's name, the strength is given by the fraction CAP - SM.
You should find the strongest extension and return a string in this
format: ClassName.StrongestExtensionName.
If there are two or more extensions with the same strength, you should
choose the one that comes first in the list.
For example, if you are given "Slices" as the class and a list of the
extensions: ['SErviNGSliCes', 'Cheese', 'StuFfed'] then you should
return 'Slices.SErviNGSliCes' since 'SErviNGSliCes' is the strongest
extension
(its strength is -1).
Example:
for Strongest_Extension('my_class', ['AA', 'Be', 'CC']) == 'my_class.AA'
```

Python:

```
    strong = extensions[0]
    my_val = len([x for x in extensions[0] if x.isalpha() and x.isupper()]) -
len([x for x in extensions[0] if x.isalpha() and x.islower()])
    for s in extensions:
        val = len([x for x in s if x.isalpha() and x.isupper()]) - len([x for
x in s if x.isalpha() and x.islower()])
        if val > my_val:
            strong = s
            my_val = val

    ans = class_name + "." + strong
    return ans
```

C++:

```
#include<stdio.h>
#include<math.h>
#include<vector>
#include<string>
#include<algorithm>
using namespace std;
#include<stdlib.h>
string Strongest_Extension(string class_name,vector<string> extensions){
    string strongest="";
```

```
    int max=-1000;
    for (int i=0;i<extensions.size();i++)
    {
        int strength=0;
        for (int j=0;j<extensions[i].length();j++)
        {
            char chr=extensions[i][j];
            if (chr>=65 and chr<=90) strength+=1;
            if (chr>=97 and chr<=122) strength-=1;
        }
        if (strength>max)
        {
            max=strength;
            strongest=extensions[i];
        }
    }
    return class_name+'.'+strongest;
}
```